Assignment 2 - Git & GitHub Fundamentals
Expense Tracker — Group Project Report

- **Group Size:** 3 students
- **Chosen Theme:** Expense Tracker
- **Full Names:** Mouaad Narjis; Sara Maskour; Razane Souane
- **Student IDs:** 159916; 159349; 163671
- **Course Codes & Sections:** CSC332402 ; CSC332402,CSC332401
- **GitHub Usernames:** Mn99971; saramaskour; razanesouane
- **GitHub Repository URL:** https://github.com/Mn99971/expense-tracker-group-b1
- **GitHub Pages URL:** https://mn99971.github.io/expense-tracker-group-b1/

1. Project & Role Distribution

Briefly describe the chosen theme, the feature areas, and who was responsible for each one. State whether the group followed the 2-member or 3-member path.

Our three-person group created a Python command-line Expense Tracker that stores expense records in a local JSON file. Mouaad developed Expense Management (add, view, edit, delete and persistence) and the CLI. Sara developed Expense Analysis (total spending, spending by category and largest expense) and the GitHub Actions test workflow. Razane developed Budget Management (set a budget, remaining balance and over-budget checks) and the GitHub Pages project page.

2. Branches, Commits & Contributions

For each member, identify the main feature branch, summarize the work completed, and mention the meaningful commits made.

Mouaad: feature/expense-management, expense logic, CLI, JSON storage and validation; later feature/cli-integration for combined menus and ID validation. Sara: feature/expense-analysis, analysis functions, tests and CI workflow (six commits observed during review). Razane: feature/budget-management, budget functions (set_budget, remaining_budget, is_over_budget), unit tests and GitHub Pages (three initial commits); feature/readme-conflict-razane for the merge-conflict exercise.

3. Pull Requests & Code Review

Identify each member's Pull Request and review responsibility. Describe the Request changes cycle and how the author updated the same branch before approval.

PR #1 introduced Expense Management. Sara requested a top-of-file module docstring and safe loading of empty/corrupted JSON data; Mouaad addressed the review on his feature branch. PR #2 introduced analysis and CI. Mouaad requested stricter amount validation and a dependency-install step; Sara pushed fixes and additional tests to the same branch, and green checks were later observed. PR #3 introduced Budget Management and the project website. Sara approved it while also noting a NaN validation issue, which Mouaad arranged to correct. PR #4 added the shared README status line. Sara also approved PR #5 (CLI integration) and PR #6 (launch instructions), and merged PR #2 and PR #4 after approval (See Figures 1 and 2.).

4. Merge Conflict

Explain how the required README conflict was created, what Current and Incoming represented in your case, and how the group decided the final resolved content.

The README conflict occurred after the project status was edited on competing branches. The changes involved “Budget feature complete.” and “Project status: Development in progress.” Razane merged main into feature/readme-conflict-razane and resolved the conflicting README.md content, preserving both status statements in the final file. In the conflict editor, Current represented Razane’s branch and Incoming represented main. The resolved change was then reviewed and merged through PR #7 (Figures 6 and 7).

5. Unit Tests

Summarize what your tests verify. Identify examples of normal behavior, an edge/boundary case, and invalid input/error handling.

Unit tests use pytest. Expense Management tests cover record creation, modification, deletion, validation and duplicate-ID rejection. Analysis tests cover spending totals, category totals and largest expense, including an empty-list edge case; numeric strings and NaN are rejected as invalid amounts. Budget tests cover setting valid budgets, remaining and over-budget calculations, and invalid inputs. Sara’s test_analysis.py contains seven tests. On the final merged main branch, python -m pytest -v reported 16 passed (Figure 3).

6. Continuous Integration

Briefly explain when your GitHub Actions workflow runs, what its job/runner/steps do, one failed CI run, the cause of the failure, and how the group fixed it.

The workflow in **.github/workflows/tests.yml** triggers on pushes and Pull Requests. It uses an Ubuntu runner, checks out code, configures Python 3.12, installs packages from requirements.txt, and runs **python -m pytest -v**. The first run (commit 8728a03) failed at the "Run tests" step with the error "No module named pytest": the first version of the workflow had no step to install dependencies and requirements.txt was empty, so the runner had no pytest. Sara added pytest to requirements.txt and an "Install dependencies" step (python -m pip install -r requirements.txt) in commit 2bfb178, and the following runs on the same branch passed (runs #3 and #4). Mouaad's review comment on PR #2 asked for the same explicit install step (See Figures 4 and 5.).

7. Ruleset & Protected main

Describe the final main ruleset. Explain which review and CI requirements had to be satisfied before a Pull Request could be merged.

The main branch uses an active GitHub ruleset requiring changes through Pull Requests and at least one approving review before merging. Force pushes and branch deletion are restricted. These settings blocked Pull Requests while review requirements remained unmet. GitHub Actions ran automated tests on pushes and Pull Requests, and the team checked the results. However, “Require status checks to pass” was not enabled in the ruleset, so passing CI was not a mandatory condition for merging (Figure 8).

8. Continuous Deployment

Explain how the GitHub Pages site is published and describe the final change that was merged and then observed on the live site.

The project website is hosted using GitHub Pages and published from the main branch. The website documents the application's features, team members, installation steps and usage instructions. Following the merge of PR #6, the live website displays the updated launch command, python main.py, demonstrating that changes merged into main are reflected on the published site.

9. Contribution Summary

For each member, list: feature(s), branch(es), Pull Request(s) authored, Pull Request(s) reviewed, and test contribution. Keep this concise.

Mouaad: Expense Management; feature/expense-management; PR #1; reviewer of PR #2; Expense Management tests, CLI integration, repo/ruleset setup and Pages activation. **Sara:** Expense Analysis and CI; feature/expense-analysis; authored PR #2 and PR #4; reviewed PR #1 (Request changes, then approved), PR #3, PR #5 and PR #6; analysis tests (test_analysis.py, 7 tests). Razane: Budget Management, GitHub Pages site and Git conflict exercise; branches feature/budget-management and feature/readme-conflict-razane; authored PR #3 and PR #7; reviewed PR #2 (line comment on tests.yml about duplicate workflow runs on push and pull_request); budget tests (test_budget.py) and the project page (index.html).

10. Fair Use of AI

Write one short paragraph following the assignment instructions. If different members used AI differently, identify those differences.

AI assistance was used by Mouaad to clarify GitHub collaboration and review steps, suggest validation/test cases, assist with CLI integration, and draft this report. Suggestions were compared with actual code and GitHub state and then checked through tests or manual review. Sara used Claude to learn the Git and GitHub workflow (cloning, branches, commits, pull requests, reviews and merge conflicts) and to draft analysis.py, the pytest tests and the GitHub Actions workflow. She ran the tests locally, read the CI logs, changed the code after Mouaad's review, and checked everything through tests and the GitHub results. Razane used AI assistance to help construct unit test cases for budget management, debug GitHub Pages deployment settings, and format project documentation. Each member remains responsible for understanding, verifying and explaining the contributions attributed to them.

11. Sources & Reflection

List useful sources used by the group. Then write 3–4 sentences on the hardest collaboration/automation problem you encountered and what your group would check first if a Pull Request, merge, test, or workflow behaves unexpectedly.

Sources: Assignment 2 - GitHub Collaboration & Automation handout; group GitHub repository and PR discussions; GitHub Actions checks; pytest tests; GitHub Pages site.

Coordinating work across three feature branches was difficult because reviews and corrections had to be completed before merges. Specific line comments helped resolve validation problems and ensure CI installed its dependencies. We also learned that passing tests alone do not remove required PR review or conversation-resolution blocks. If a PR or workflow behaved unexpectedly, we would first inspect the branch diff, review status, failing check logs and active branch ruleset.

Evidence (Screenshots):

Figure 1 (Section 3): Sara's Request changes review and inline comment on PR #1.

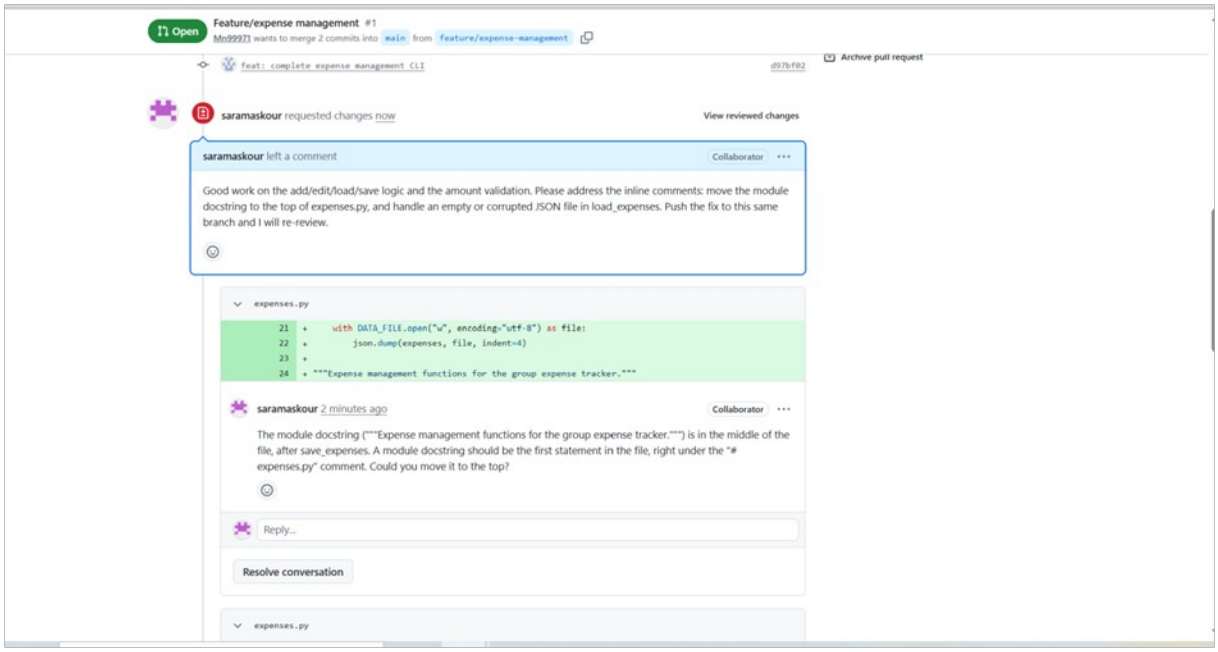


Figure 2 (Section 3): Mouaad's review comments on PR #2 and Sara's reply after fixing the code on the same branch.

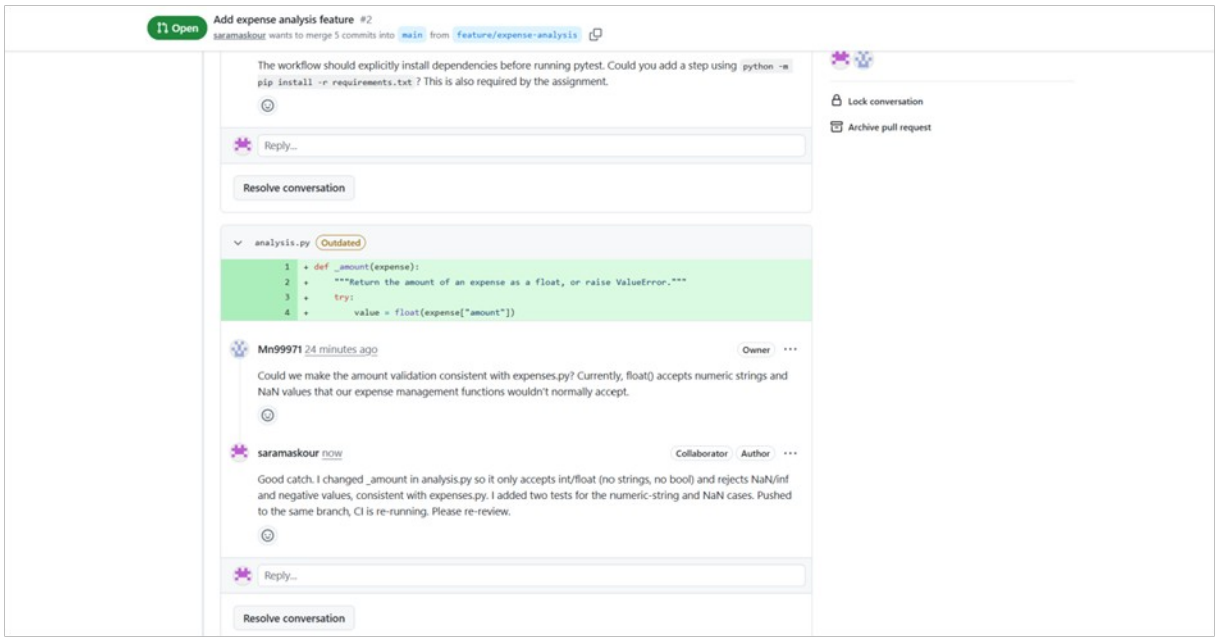


Figure 3 (Section 5): Final pytest run on main, 16 passed.

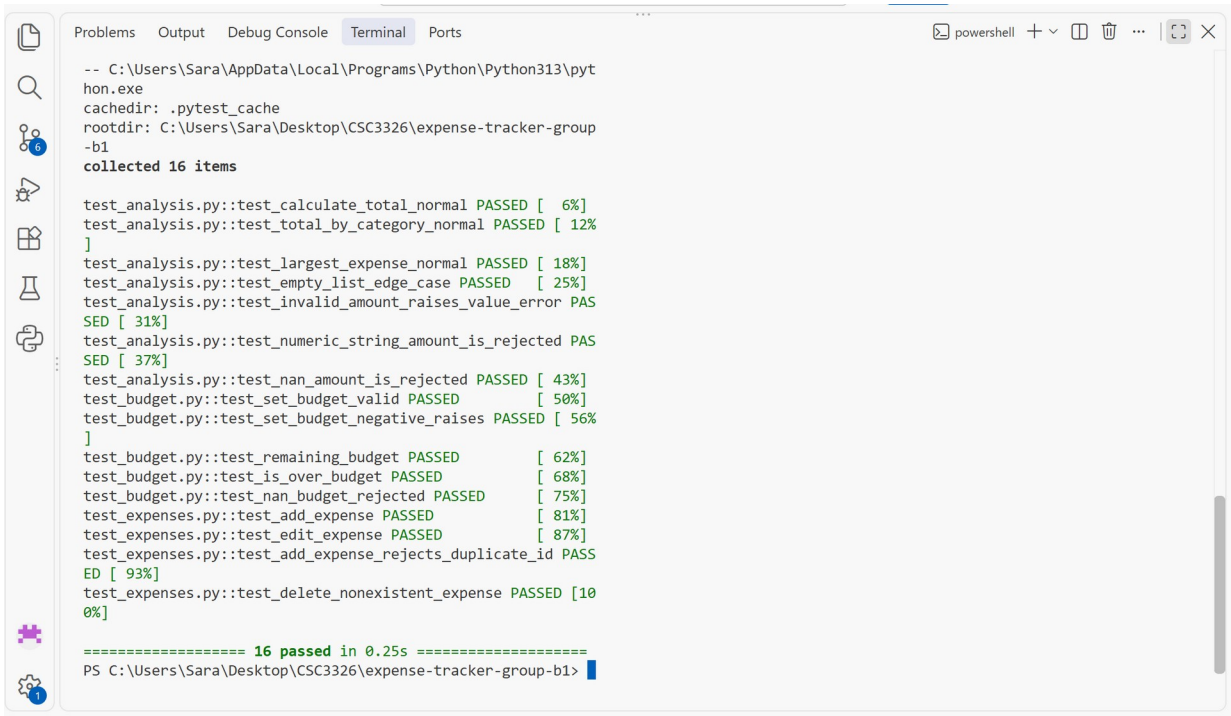


Figure 4 (Section 6): GitHub Actions run #1 failed with "No module named pytest" (Sara).

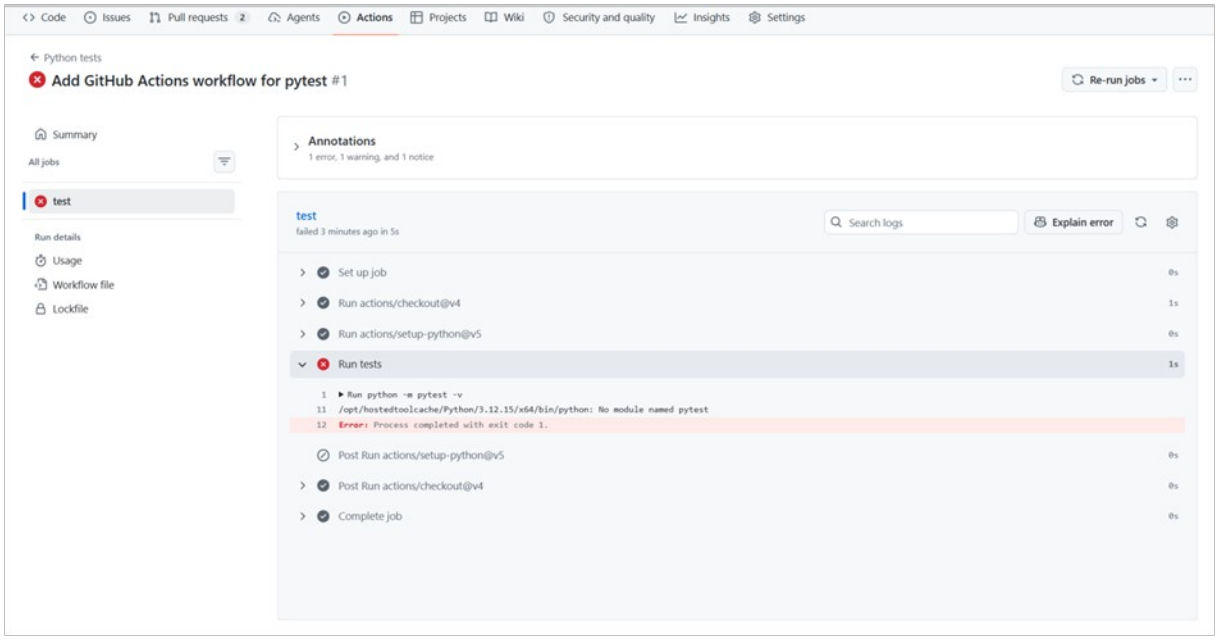


Figure 5 (Section 6): GitHub Actions run #3 passed after commit 2bfb178 (Sara).

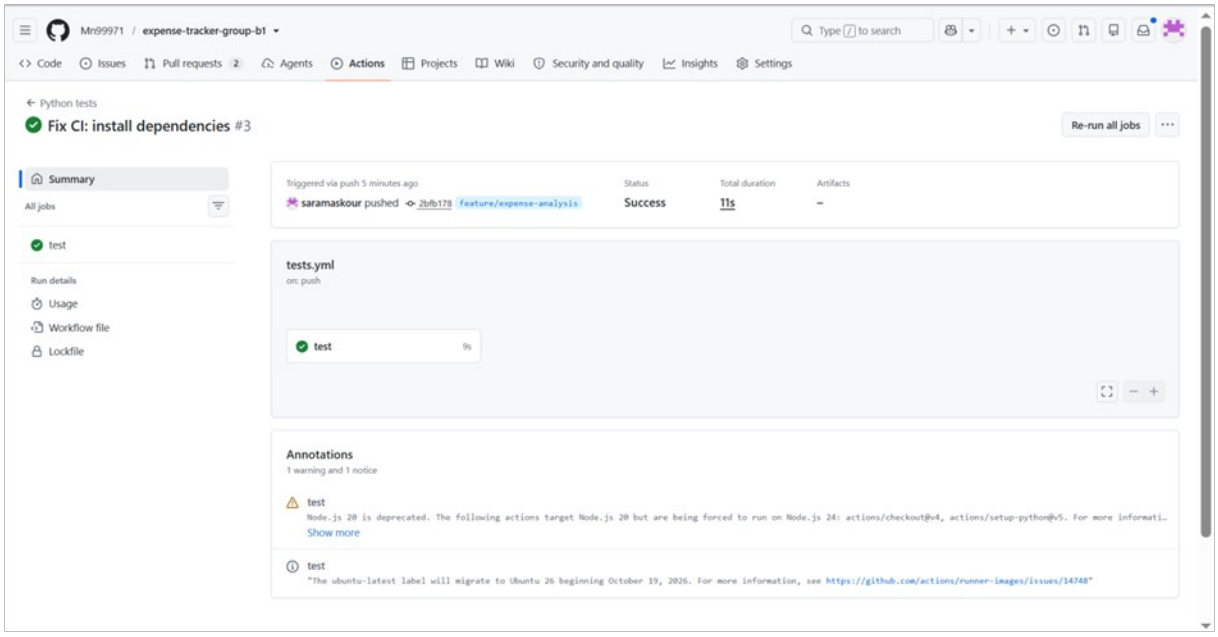


Figure 6 (Section 4): README merge conflict markers (Razane).

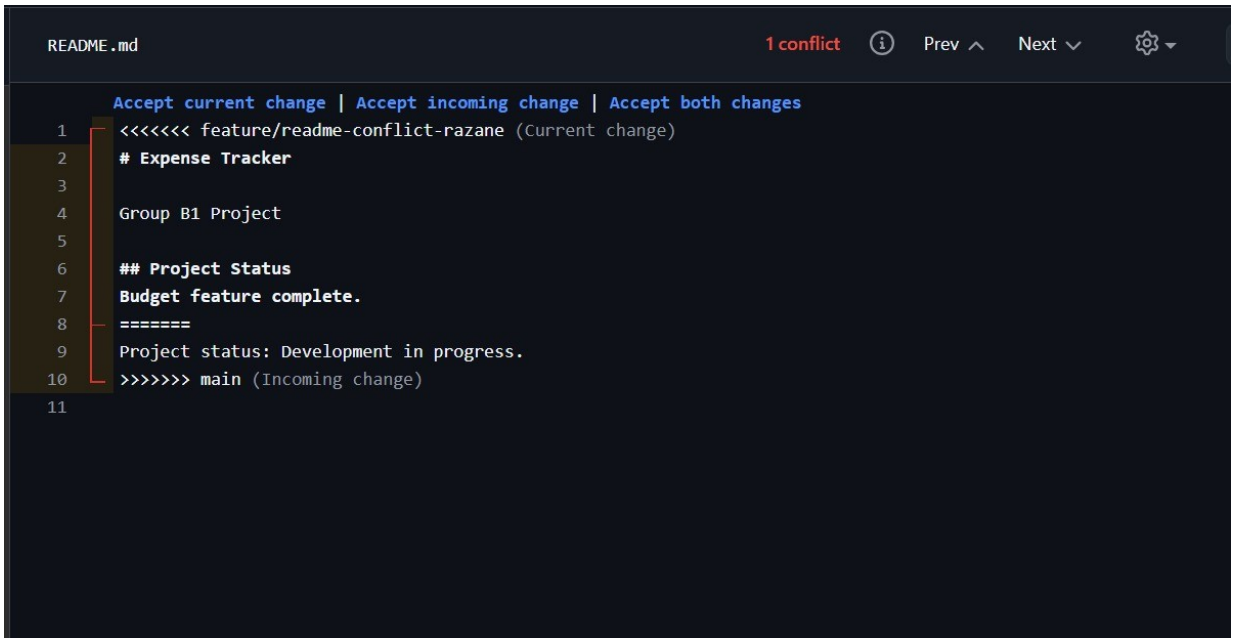


Figure 7 (Section 4): README conflict resolved (Razane).

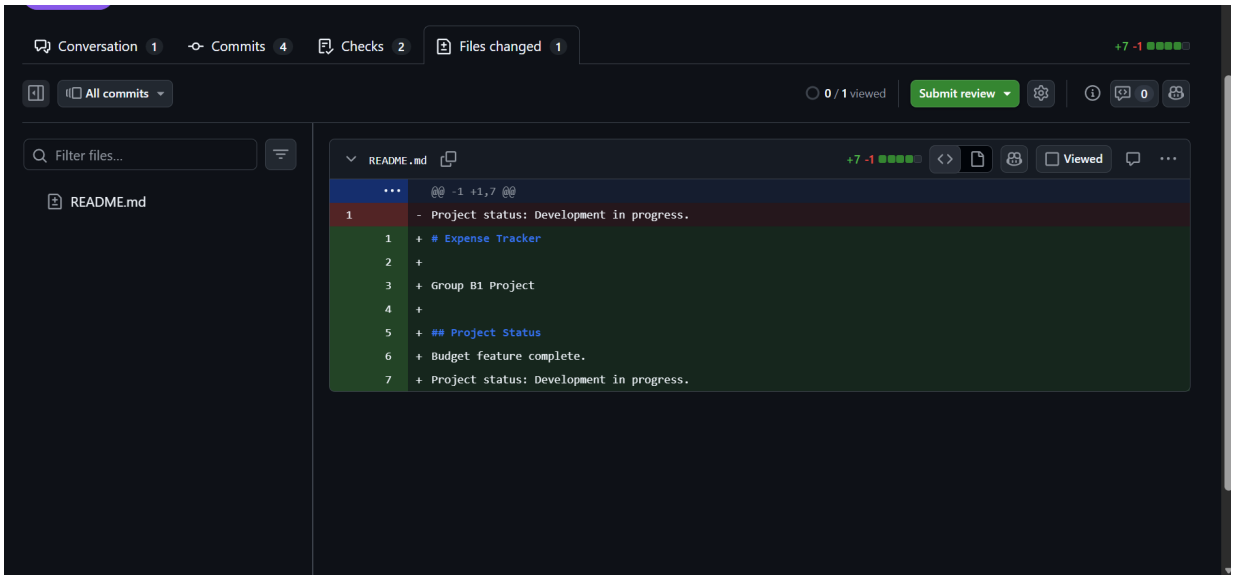
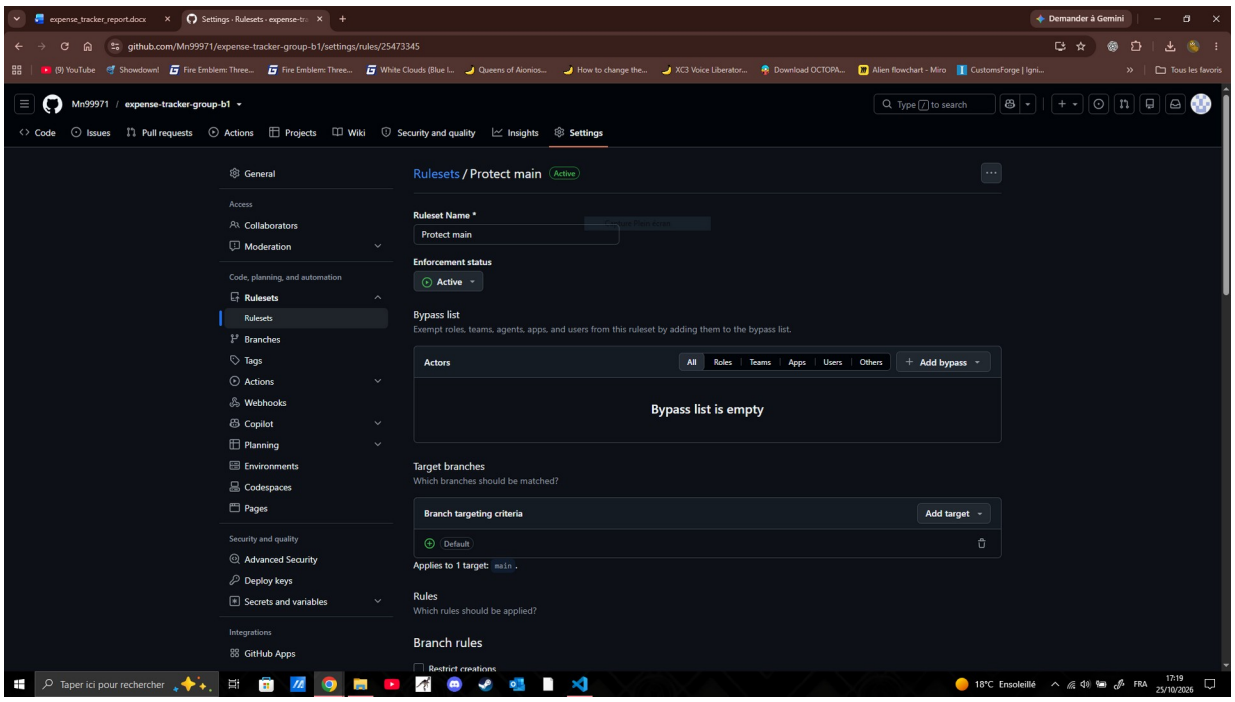


Figure 8 (Section 7): Active ruleset protecting main; Pull Requests required, but passing status checks not required (Mouaad).



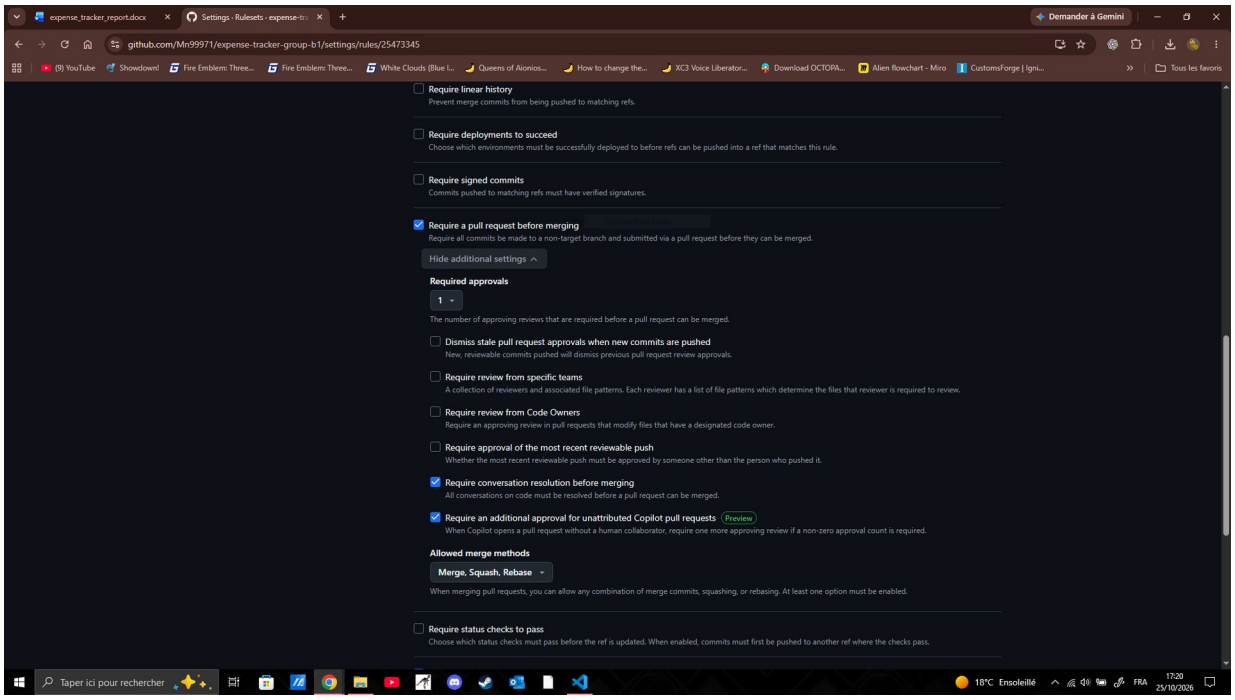


Figure 9 (Section 8): Live GitHub Pages site before and after the launch-instructions update (Mouaad / Razane).

